

Daily Bugle Walkthrough — TryHackMe

Title: TryHackMe — Daily Bugle Walkthrough

Author: Leevan Michael Vaz

Target: Daily Bugle (TryHackMe)

Target IP: 10.49.137.140

Difficulty: Hard

Category: Web Application Exploitation, Linux Privilege Escalation

Introduction

Daily Bugle is a Linux-based TryHackMe room built around a vulnerable Joomla! CMS installation. The objective is to gain an initial foothold on the web server, escalate to a local user via credential reuse, and finally obtain root through a sudo misconfiguration. The path to root touches four distinct skill areas: web enumeration, CMS vulnerability research, password cracking, and Linux privilege escalation via GTFOBins — which makes it a solid end-to-end exercise in the standard penetration testing lifecycle.

Objective: Gain initial access to the target machine, obtain the user flag, escalate privileges to root, and retrieve the root flag.

Methodology

The engagement followed a standard black-box methodology:

1. **Reconnaissance** – Port and service discovery with Nmap.
2. **Enumeration** – Web content discovery and CMS fingerprinting.
3. **Vulnerability Research** – Mapping the identified CMS version to known CVEs.
4. **Exploitation** – Weaponizing the identified SQL injection vulnerability to extract credentials.

5. **Post-Exploitation** – Leveraging admin access to achieve remote code execution.
 6. **Privilege Escalation** – Pivoting from a web-server user to a local Linux user, then to root.
-

Enumeration

Port and Service Discovery

An initial Nmap scan was run against the target's most common ports using version detection and default scripts:

```
nmap -sC -sV -p- 10.49.137.140
```

The scan revealed two open services:

Port	Service	Notes
22	SSH	OpenSSH
80	HTTP	Web application present

Web Content Discovery

With only a web server and SSH exposed, port 80 became the primary attack surface. Directory brute-forcing was performed with Gobuster/Dirsearch against the target:

```
gobuster dir -u http://10.49.137.140 -w /usr/share/wordlists/dirbuster/directory-list-2.3-medium.txt
```

This surfaced several directories and files consistent with a Joomla! installation, most notably:

- `/administrator` — the Joomla admin login panel
- Various Joomla core directories (`/components` , `/templates` , `/media` , etc.)
- `configuration.php` — Joomla's main configuration file, which normally holds database credentials

CMS Fingerprinting

The presence of `/administrator` and Joomla-specific paths, combined with the page source and a `WhatWeb` scan, confirmed the target was running a Joomla! CMS. The exact version was pinned down by requesting the `administrator/manifests/files/joomla.xml` file, a common technique for version fingerprinting when the "generator" meta tag has been stripped from the front end. This confirmed the installation as:

```
Joomla! 3.7.0
```

Vulnerability Research

With the CMS version confirmed, the next step was to check for known, unauthenticated vulnerabilities affecting Joomla 3.7.0:

```
searchsploit joomla 3.7.0
```

This returned a match for a SQL injection vulnerability in the `com_fields` component:

- **CVE-2017-8917** — Joomla! `com_fields` Component SQL Injection

This is a pre-authentication, blind SQL injection reachable through the `list[fullordering]` parameter of the `com_fields` endpoint:

```
http://10.49.137.140/index.php?option=com_fields&view=fields&layout=modal&list[fullordering]=<injection>
```

Because the component is publicly reachable without authentication, this represented a direct path to the Joomla database.

Exploitation

Automated Exploitation with joomblah

Rather than manually building blind SQLi payloads, the publicly available `joomblah.py` exploit script was used to automate detection and extraction. This script targets the same `com_fields` SQL injection and automatically dumps the Joomla users table.

```
python3 joomblah.py http://10.49.137.140
```

The exploit successfully identified the injection point and dumped the contents of the Joomla users table (`fb9j5_users` in this instance), returning:

Field	Value
Username	<code>jonah</code>
Email	<code>jonah@tryhackme.com</code>
Password hash	bcrypt hash

Cracking the Password Hash

The extracted bcrypt hash was saved to a file and attacked offline with John the Ripper using the `rockyou.txt` wordlist:

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

The hash cracked successfully, recovering the plaintext credentials:

```
Username: jonah  
Password: spiderman123
```

Gaining Administrative Access

These credentials were used to authenticate to the Joomla administrator panel:

```
http://10.49.137.140/administrator
```

Login succeeded, granting full Joomla Administrator privileges — effectively converting a SQL injection into complete control of the CMS.

Post-Exploitation

Achieving Remote Code Execution via the Template Editor

Joomla administrators can edit template files directly through the web UI, which is a well-known route to remote code execution once admin access is obtained. The path taken was:

```
Extensions → Templates → Templates → Protostar → index.php
```

The `index.php` file of the active (Protostar) template was edited to include a PHP reverse shell payload at the top of the file, configured with the attacker's VPN IP address and a listening port.

A Netcat listener was started to catch the callback:

```
nc -lvnp <port>
```

Browsing to the site's homepage triggered the malicious template code, and the listener caught a reverse shell running as the low-privileged `apache` user.

Local Enumeration

With a shell as `apache`, standard post-exploitation enumeration was carried out. Reviewing `/etc/passwd` revealed two accounts of interest:

- `root`
- `jjameson`

Since Joomla's `configuration.php` file typically stores the database connection string in plaintext, it was inspected next:

```
cat /var/www/html/configuration.php
```

This exposed a database password:

```
nv5uz9r3ZEDzVjNu
```

Lateral Movement to a Local User

Credential reuse is common on CTF-style boxes, so the recovered database password was tried against the `jjameson` account over SSH:

```
ssh jjameson@10.49.137.140
```

The password matched, granting an interactive shell as `jjameson`. From here, the user flag was retrieved:

```
27a260fe3cba712cfdedb1c86d80442e
```

Privilege Escalation

Enumerating Sudo Rights

The first privilege escalation check performed as `jjameson` was:

```
sudo -l
```

This showed that `jjameson` could run `/usr/bin/yum` as root without a password:

```
(root) NOPASSWD: /usr/bin/yum
```

Exploiting the Yum Misconfiguration (GTFOBins)

`yum` is a documented GTFOBins privilege escalation vector: because it can be configured to run arbitrary plugins or install packages, an attacker with sudo rights over `yum` can use it to execute commands as root, typically by crafting a malicious RPM package (or an equivalent plugin-based technique) that runs an arbitrary command during installation.

Following the GTFOBins method for `yum`, a root shell was obtained. Root access was confirmed with `id`, and the root flag was retrieved from `/root/root.txt`.

Findings Summary

#	Finding	Impact
1	Joomla 3.7.0 vulnerable to CVE-2017-8917 (SQL injection)	Full database compromise, credential disclosure, admin account takeover
2	Weak administrator password	Cracked in seconds using John the Ripper + rockyou.txt
3	Sensitive credentials stored in <code>configuration.php</code>	Database password reuse enabled SSH access as a local Linux user
4	Improper sudo permissions on <code>yum</code>	Full system compromise via GTFOBins

MITRE ATT&CK Mapping

Tactic	Technique
Reconnaissance	Active Scanning (T1595)
Discovery	Software Discovery (T1518)
Initial Access	Exploit Public-Facing Application (T1190)
Credential Access	Password Cracking (T1110.002)
Credential Access	Credentials from Configuration Files (T1552)
Persistence	Web Shell (T1505.003)
Lateral Movement	Valid Accounts (T1078)
Privilege Escalation	Abuse Elevation Control Mechanism (T1548)
Execution	Command and Scripting Interpreter (T1059)

Conclusion

Daily Bugle demonstrates a realistic attack chain: an unpatched, publicly known CMS vulnerability leads to full application compromise, weak credential hygiene enables lateral movement to the underlying OS, and an overly permissive sudo rule finishes the job by granting root. No single failure was catastrophic on its own — it was the combination of an outdated CMS, password reuse, and excessive sudo privileges that turned a web application flaw into full system compromise.

Mitigation Recommendations

- Upgrade Joomla to the latest supported version and apply the patch for CVE-2017-8917.
- Remove unused or default administrator accounts.
- Enforce strong password policies and prevent password reuse across services and accounts.
- Restrict filesystem access to `configuration.php` and avoid storing secrets in plaintext where possible.
- Apply the principle of least privilege to sudo rules; remove unnecessary rights such as passwordless `yum` execution.
- Monitor administrative login activity and alert on anomalous access patterns.
- Conduct regular vulnerability assessments and CVE tracking for all deployed CMS platforms.